

AI-ENABLED SMART

CAMPUS

WITH WIRELESS SENSOR NETWORK



- **Group ID:** BCAIAA22
- **Team:** Aditya Nalge
Aryan Dhondage
Aditya Bade
Om Teke
- **Faculty Guide:** Dr. Tushar Mote



THE PROBLEM & USER NEED

-Campus energy is difficult to manage when information arrives only after the bill

- Classrooms, labs, libraries and server rooms consume substantial electricity.
- Monthly bills and manual meter readings show totals—not where, when or why energy was used.
- Without a live baseline, staff cannot easily compare today's usage with yesterday's or semester trends.
- Faulty equipment and lights or fans left running may remain unnoticed.
- Fixed thresholds can trigger repeated false alarms during normal changes in weather or activity.

OBJECTIVES & PROPOSED SOLUTION



- CONTINUOUSLY MONITOR VOLTAGE, CURRENT, POWER AND ENERGY FOR INDIVIDUAL LOADS.
- TRANSMIT SENSOR READINGS WIRELESSLY TO A CENTRAL BACKEND AND STORE TIMESTAMPED HISTORY.
- FORECAST ENERGY CONSUMPTION FOR THE NEXT 10-MINUTE INTERVAL USING MACHINE LEARNING.
- DETECT UNUSUAL CONSUMPTION FROM PREDICTION ERRORS AND LOG ALERTS FOR REVIEW.
- PRESENT CURRENT READINGS, TRENDS, PREDICTIONS AND ANOMALY HISTORY IN ONE DASHBOARD.
- KEEP THE DESIGN LOW-COST AND SCALABLE FROM A TEST BENCH TO MULTIPLE CAMPUS BUILDINGS.

END-TO-END SYSTEM ARCHITECTURE

OPERATIONAL SEQUENCE



- FastAPI receives each reading, timestamps and stores it, then returns prediction and anomaly status.
- Thee AI model is loaded once at startup; XGBoost inference is reported under 2 ms.
- Five REST endpoints expose sensor ingestion, current status, history, predictions and anomaly events.

HARDWARE & MEASUREMENT WORKFLOW

WIRING / DATA

INA219 sensors use I²C: SDA GPIO21, SCL GPIO22. DHT11 data connects to GPIO4.

FLOW

Sensors → ESP32 → Wi-Fi → backend → database → dashboard

CORE COMPONENTS

- ESP32: WI-FI-ENABLED MICROCONTROLLER COORDINATING SENSING AND CONTROL.
- 2 × INA219: MEASURE VOLTAGE AND CURRENT FOR THE LED AND FAN CIRCUITS; POWER IS CALCULATED FROM READINGS.
- DHT11: RECORDS TEMPERATURE AND HUMIDITY TO ADD ENVIRONMENTAL CONTEXT.
- 2 × MOSFETS: SWITCH THE LED AND FAN LOADS UNDER ESP32 CONTROL.
- 1N4007 DIODE: SUPPRESSES FAN SWITCHING VOLTAGE SPIKES.
- SEPARATE 5 V DC LOAD SUPPLY KEEPS LED/FAN CURRENT AWAY FROM THE ESP32 SUPPLY.

DATASET, CLEANING & FEATURE ENGINEERING

1. Data Cleaning: Arranged data by date and time and removed unnecessary columns.

2. Handling Anomalies: Kept unusually high energy values because they may indicate abnormal energy consumption.

3. Feature Engineering: Created features using past energy readings, averages, time, temperature, humidity, and lighting data.

4. Preventing Data Leakage: Used only past data for future predictions and removed the first rows that lacked sufficient historical data.

FORECASTING MODELS & EVALUATION

TIME-AWARE VALIDATION

Chronological split: 70% train, 15% validation, 15% test. Random splitting was avoided to reduce leakage.

MODEL DECISION

1XGBoost is compact (~200 KB), fast and more explainable. LSTM is retained as a research reference.

MODEL	MAE (Wh)	R ²	INFERENCE	ROLE
Naive baseline	49.9	−0.01	<1 ms	Minimum benchmark
Linear Regression	44.3	0.20	<1 ms	Interpretable baseline
Random Forest	37.6	0.30	~15 ms	Nonlinear model; large file
XGBoost (tuned)	34.1	0.35	<2 ms	Selected for production
LSTM	23.7	0.58	~90 ms	Most accurate tested

ANOMALY DETECTION & BACKEND SERVICES

FASTAPI ENDPOINTS

EXAMPLE

Actual 800 Wh – predicted 200 Wh = +600 Wh residual. Since +600 Wh exceeds +219.5 Wh, the event is flagged as a high anomaly.

ENDPOINT	PURPOSE
POST /api/sensor-data	Store reading; run forecast and anomaly check
GET /api/current	Latest reading and prediction
GET /api/history	Historical readings aggregated by hour/day
GET /api/prediction	Fresh on-demand prediction
GET /api/anomalies	Anomaly events; filter by confirmation status

DATA PLATFORM, DASHBOARD & FEASIBILITY

Target environments:

classrooms
laboratories
libraries
server rooms

- Scalable architecture supports adding sensor nodes and rooms.
- Dashboard replays 1,008 test-set readings at 500 ms intervals.
- Current evidence is a prototype; real deployment requires calibration, security hardening and live-data validation.

DATABASE

sensor_readings, predictions and anomalies; SQLite in development, PostgreSQL for multi-classroom production.

DASHBOARD

Chart.js dashboard: live readings, actual vs predicted chart, hourly/daily totals, severity and anomaly event log.

DEPLOYMENT

Model file: xgboost_tuned.json; thresholds in anomaly_thresholds.npy; LSTM kept as a reference.

CONCLUSION, LIMITATIONS & NEXT STEPS

The project provides an end-to-end smart classroom energy monitoring system using wireless sensors, ESP32, machine learning, and a web dashboard. It can monitor energy usage, forecast short-term consumption, and identify unusual readings. XGBoost offers a practical balance of speed, accuracy, and explainability for deployment.

Limitations

- The training dataset comes from household energy usage, not real classrooms.
- Occupancy, fan usage, projector usage, and class schedules were not included.
- Anomalies were identified statistically because labelled fault data was unavailable.
- The model requires testing and retraining with real campus data.
- Current results represent a prototype rather than a production deployment.

Next Steps

- Collect at least 90 days of real ESP32 sensor data.
- Retrain and compare the machine-learning models using classroom conditions.
- Add occupancy, light, temperature, and air-quality sensors.
- Validate alerts against room schedules and actual equipment faults.
- Improve security and deploy the system to a cloud or mobile platform.
- Explore TinyML for faster, low-cost edge-based predictions.